Session Types, Simply

Nov 7, 2026 · @Robert Jason Fielding

Session types fit in six ideas every engineer already uses: grammars, state machines, queues, and yes-or-no questions with evidence. This page explains them that way, then maps each idea back to the textbook terms.

Why session types

The worst bugs between services sit at the boundary, not inside any one service. A message arrives that the receiver has no handler for. A service waits forever for a reply nobody owes it. Two services each think the other one decides.

Without a contract, the only way to know what the other side will send is to read its code. A session type is that contract: the whole conversation, written once, from nobody's point of view. Each party then checks its own part against it, without reading anyone else's code.

The idea is about 30 years old and still rare in industry. The usual explanations need a process calculus, typing judgments and a projection operator before the first useful example. None of that machinery is needed to use the idea.

The textbook version, in one paragraph

You will meet this elsewhere, so here it is once. In multiparty session types (Honda, Yoshida and Carbone, 2008), a *global type* such as $p \to q : \ell(S).G$ describes the whole conversation. *Projection*, a partial function with a *merge* operator for branches, derives one *local type* per role, built from sends ($q!\ell$) and receives ($p?\ell$). Programs are written in a typed π-calculus, and a typing judgment checks each process against its local type. *Subject reduction* then yields three theorems: communication safety, session fidelity and progress. Channels are asynchronous, unbounded and lossless. Deciding when one local type may replace another (asynchronous subtyping) is undecidable in general.

Nothing below depends on this paragraph. The table near the end maps each of its terms to the plain version.

1. A protocol is a grammar whose words are messages

A grammar lists the legal sentences of a language. A protocol lists the legal conversations of a system. The only difference is the words: in a protocol, each word is one message, with a sender and a receiver.

Here is a purchase between a client, a shop and a bank, in LeanFM's notation:

```
protocol Purchase (Client, Shop, Bank) {
  o: Order from Client to Shop;
  choice {
    q: Quote from Shop to Client cause o;
    p: Pay   from Client to Bank cause q where p.amount = q.price;
       Paid  from Bank to Shop cause p;
       Shipped from Shop to Client;
  } or {
       SoldOut from Shop to Client cause o;
  }
}
```

It has exactly two legal conversations: Order, Quote, Pay, Paid, Shipped; or Order, SoldOut. The constructs are the ones every grammar has:

- **Sequence:** one line after another.
- **Choice:** `choice { … } or { … }`; exactly one branch happens.
- **Repetition:** a named block that `continue` repeats.
- **Parallel:** `par { … } and { … }`; both happen, interleaved.

Two additions carry data. `cause q` says this message answers `q`. `where p.amount = q.price` is a condition on payloads; it is how the grammar ties a later message to an earlier one.

That is the whole of what the textbook calls a global type. A recorded run of the real system is correct when it is a sentence of this grammar, which can be checked from logs alone.

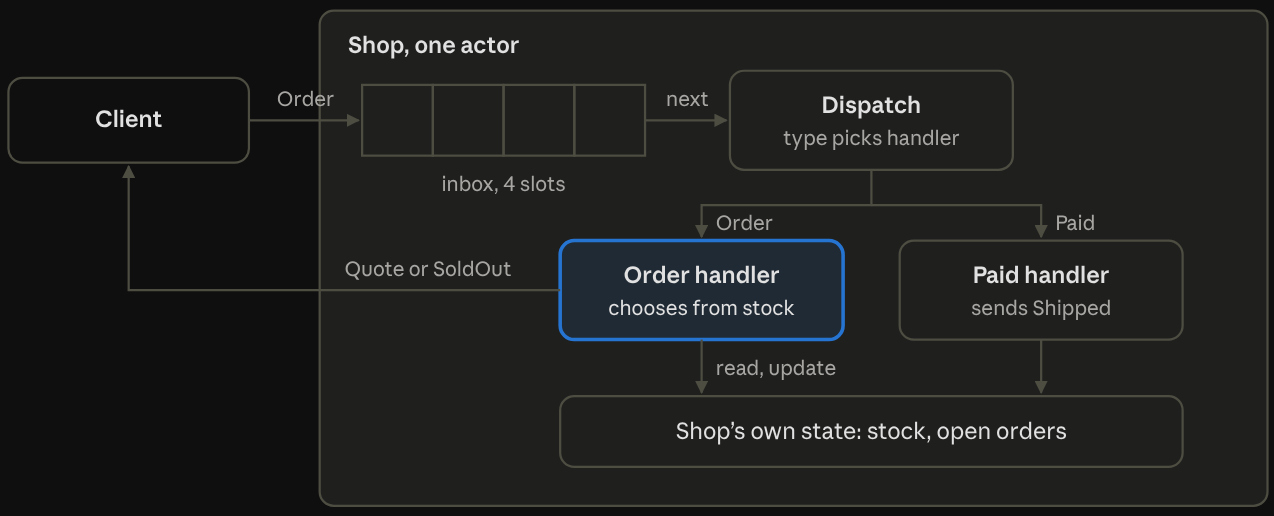
2. Every party is a state machine that reacts

Each party in the conversation is an *actor*: a state machine with an inbox of fixed size. It repeats one loop:

1. Take the next message from the inbox.

2. The message picks the handler. An `Order` runs the order handler; a `Paid` runs the payment handler.
3. The handler updates the actor's state, and may put messages in other actors' inboxes.

A message picks the handler; the choice uses only the shop's own state



one actor's loop · inbox, dispatch, handlers, own state

The shop never sees the client's state, only the messages in its inbox; its reply depends on its own stock.

That is how real servers, mobile apps and embedded controllers already work. It also settles who makes choices. **Every choice is made inside one actor, from what that actor has seen.** In the purchase, only the shop knows its stock, so the shop decides between Quote and SoldOut. The client cannot decide for it; the client learns the decision when Quote or SoldOut arrives.

Nothing is chosen globally. There is no shared coin that everyone sees at the same instant. An actor can only affect another by putting a message in its inbox. This is the physical rule the rest of the method depends on.

3. What each party owes is the grammar, filtered

To find what one actor must do, delete every message it neither sends nor receives. What remains is its part, written as sends and receives. The shop's part of the purchase:

```
receive Order from Client;
select {
  send Quote to Client;
  receive Paid from Bank;
  send Shipped to Client;
} or {
  send SoldOut to Client;
}
```

The bank's part:

```
offer {
  receive Pay from Client;
  send Paid to Shop;
} or {
}
```

The two choice words say who decides. `select` is a choice this actor makes. `offer` is someone else's choice, which this actor learns from what arrives. The bank's second branch is empty: when the shop sells out, the bank is never involved.

The textbook calls this *projection*, and each filtered part a *local type*. In practice it is the handler code a team owes for one conversation, and the test it must pass.

The two rules that make a protocol buildable

Not every grammar can be built from actors that only see their own inboxes. Two checks rule out the ones that cannot. Both follow from idea 2.

Rule 1: one actor starts every branch of a choice. That actor is the one deciding. A first draft of a file transfer lets the sender's timer start one branch and the receiver's acknowledgement start the other. Both can happen at once, so the protocol is a race. LeanFM reports:

```
no single role decides a choice, so it is a race: Sender starts Timeout;
Receiver starts Ack
```

Rule 2: anyone whose future depends on the choice must be told by a message. Suppose the client first places a hold with the bank, then the shop sells out:

```
protocol PurchaseWithHold (Client, Shop, Bank) {
  h: Hold from Client to Bank;
  o: Order from Client to Shop;
  choice {
    q: Quote from Shop to Client cause o;
    p: Pay   from Client to Bank cause q;
  } or {
      SoldOut from Shop to Client cause o;
  }
}
```

The bank holds the money and never learns whether to release it. LeanFM reports:

```
Bank cannot tell which branch Shop chose
```

The plain `Purchase` passes rule 2 even though the bank is not told about SoldOut. The bank's part starts when Pay arrives; in the SoldOut branch it never starts, so it owes nothing. The textbook projection rejects `Purchase` here. This version accepts it because that is how servers work: a handler that was never invoked has nothing to clean up.

The textbook names for these are *realizability* or *non-local choice* (rule 1) and *projectability* (rule 2).

4. Channels have physics

The textbook assumes every message arrives exactly once, in order, through a queue that never fills. Real channels do none of that, so each channel's behaviour is declared alongside the protocol:

Property	What gets declared
Capacity	the queue size, and what happens when it is full: block the sender, reject, drop the newest, or drop the oldest
Delivery	ordered or not; lost, duplicated, or neither
Time	timers, which may fire before the reply they wait for has had time to arrive
Failure	lost messages, restarted actors, expired sessions

The physics changes what the same grammar means. The original TFTP sender (RFC 783) and the corrected one (RFC 1123) follow the same lock-step grammar, and both pass the textbook checks. With one early timer declared, the original resends a block, the receiver acknowledges both copies, and every later block then travels twice. This is the Sorcerer's Apprentice bug. With the same physics, the corrected sender has no such run; the check covers all 509 nodes of the model.

Bounded queues also make checking finite. Actors talking through unbounded queues can express undecidable problems (Brand and Zafiropulo, 1983). The textbook escapes this by restricting the protocol's shape. This version bounds the queues instead, as the hardware already does, so the number of states is finite.

5. Guarantees become questions with evidence

Finite actors and finite queues give a finite map of every state the system can reach. Any question about that map gets a yes or no, with a path as evidence: a witness for "this can happen", or a counterexample for "this must never happen".

The textbook's three theorems become three such questions. They are written in CTL, a standard notation for questions over branching futures: **A** means on every path, **E** on some path, **G** always, **F** eventually.

Textbook theorem	The same guarantee as a question
Communication safety	never: a message is waiting that its receiver has no handler for
Session fidelity	every run is a sentence of the grammar; also checked on recorded production logs
Progress	never: some actor waits, and nothing can ever move again, before the conversation ends

The textbook proves its theorems once, for every program that type-checks, under its channel assumptions. Here they are asked of one system with its declared physics, and any other question can be asked the same way:

- `AG (Pay → AF Shipped)`: whenever the client pays, the goods eventually ship.
- `EF SoldOut`: selling out is possible, so the test for it is not dead code.

- `AG AF Idle`: the system always returns to idle; this needs a fairness assumption, stated as a requirement rather than hidden.

A counterexample is a concrete run, which can be drawn as a sequence diagram or replayed as a test.

6. An attacker is one more actor

The textbook assumes every party is honest and the channel is private. Security needs the opposite: an actor, here called Ivo, who *is* the network. Every message an honest actor sends goes to Ivo. Every message an honest actor receives is one Ivo could build.

What Ivo can build is decided by `knows(actor, value)`. It applies the standard Dolev–Yao rules: take apart pairs, open ciphertexts you hold the key for, and build pairs, encryptions and hashes from what you hold. Each state on the map records what Ivo knows, so secrecy is just another question.

The Needham–Schroeder protocol (1978) is three messages, and it is a perfectly well-formed session type. Ask:

```
never ( claimed commit(B, A) and knows(Ivo, nb) )
```

That is: Bob must never believe he is talking to Alice while Ivo holds Bob's secret nonce. LeanFM explores 228 states and returns a 10-state counterexample: Lowe's 1995 man-in-the-middle attack. With Lowe's fix, Bob's name added inside his reply, the same question is proved over all 220 reachable states.

The same function works for defenders. A filter in front of Bob can only block what `knows` lets it see. Without Bob's private key, an encrypted `bad` goes straight past it, and the counterexample shows the filter itself delivering it.

Translation table

Textbook term	Plain version
Global type	the protocol: a grammar whose words are messages
Role	an actor: a state machine with a bounded inbox

Textbook term	Plain version
Projection	delete the messages an actor neither sends nor receives
Local type	one actor's part: its sends, receives and choices
Internal choice (⊕, select)	a choice this actor makes
External choice (&, offer, branch)	a choice another actor makes, learned from what arrives
Merge operator	rule 2: anyone whose future depends on a choice is told by a message
Realizability, non-local choice	rule 1: one actor starts every branch
Typing judgment, subject reduction	checking each actor's code against its part; checking logs against the grammar
Communication safety	never: a message waits that has no handler
Session fidelity	every run is a sentence of the grammar
Progress, deadlock freedom	never: stuck before the end
Asynchronous semantics	inboxes, with capacity and physics declared per channel
Duality (binary sessions)	for two actors, one's sends are the other's receives

What the simple version gives up

It trades generality for concreteness. Four things are lost:

- **Unbounded results.** A textbook theorem covers every number of sessions and any queue length. Here a "proved" means proved within the declared bounds: queue sizes, session counts, value domains.
- **Checking one piece at a time.** Type-checking looks at one process against its local type, so cost grows with the size of the code. Exploring the map looks at all actors together, and the number of states can grow very fast with the number of actors and the queue sizes.

- **Compile-time guarantees in a language.** Session-typed languages reject a bad program before it runs. Here conformance is checked against the protocol, a model of the code, and recorded logs. A program that never logs a bad run is not thereby proved correct.
- **General recursion.** Repetition is a tail-position loop, and data enters only through conditions on payloads. This keeps the map finite.

References

- Honda, Yoshida and Carbone, Multiparty asynchronous session types, POPL 2008; journal version in JACM, 2016.
- Brand and Zafiropulo, On communicating finite-state machines, JACM, 1983.
- Bravetti, Carbone and Zavattaro, On the boundary between decidability and undecidability of asynchronous session subtyping, 2017.
- Lowe, An attack on the Needham–Schroeder public-key authentication protocol, Information Processing Letters 56, 1995.
- LeanFM: the notation, checks and verdicts used in this page; every example output is LeanFM's own.